

Módulo 1

¿Qué es un agente de IA?

Y por qué cambia todo cuando puede leer tus datos y ejecutar código

Claude vs. Claude Code

El modelo



El sistema de inteligencia artificial desarrollado por Anthropic. Razona, escribe, analiza y responde.

- Accesible vía claude.ai (navegador), API, o herramientas construidas sobre él
- claude.ai es conversacional: solo lees y escribes texto
- No tiene acceso a tu computador ni a tus archivos
- No ejecuta código real — describe qué haría

Claude + herramientas



Claude corriendo en tu terminal, con acceso directo a tu entorno de trabajo.

- CLI que corre en la terminal — no en el navegador
- Lee y escribe archivos reales en tu disco
- Ejecuta código Python y comandos de terminal
- Ve el resultado, detecta errores y los corrige solo

Claude:

El modelo de IA → el cerebro

claude.ai:

Interfaz web para chatear con Claude

Claude Code:

Claude en tu terminal con acceso a tu computador

Chatbot vs. Agente

¿Cuál es la diferencia?



Chatbot

- Responde texto
- No accede a archivos
- No ejecuta código
- No corrige sus errores solo
- Cada conversación es aislada



Agente

- Lee archivos de tu disco
- Ejecuta código real
- Ve el resultado y lo corrige
- Hace planes antes de actuar
- Tiene memoria de sesión

Cómo trabaja Claude Code

El ciclo acción → observación → corrección



El modelo mental del curso

Dos roles distintos, un solo objetivo científico

Tú (científico/a)	Claude Code (agente)
Entiendes el modelo biológico	Implementa el algoritmo
Sabes qué restricciones termodinámicas importan	Escribe y corre el código Python
Evalúas si el flujo metabólico tiene sentido	Reporta resultados y corrige errores
Defines las condiciones del modelo	Genera figuras y tablas
Escribes el CLAUDE.md	Lee archivos, ejecuta Python

Demo en vivo

Claude Code lee el paper de Salvy et al. (2019) y propone la reimplementación

Paper · Salvy et al. (2019) · *Bioinformatics* 35(1):167–169

pyTFA and matTFA: a Python package and a Matlab toolbox for Thermodynamics-based Flux Analysis

Repositorio original (EPFL-LCSB/pytfa) sin mantenimiento activo desde 2021 — incompatible con Python 3.10+ y versiones actuales de COBRApy

El agente va a:

- Leer el PDF y el código del repositorio
- Identificar qué partes del código están obsoletas
- Proponer un plan de reimplementación modular
- Señalar qué dependencias deben actualizarse

Tu rol durante la demo

- ¿Entendió el algoritmo TFA?
- ¿El plan tiene sentido para modernizar el código?
- ¿Detectó los problemas de compatibilidad?

El dataset que trabajaremos

Neilson et al. (2017) — mSystems — Desierto de Atacama, Chile

Sitio de estudio

Dos transectos: Baquedano (BAQ) y Yungay (YUN)

Diseño

16 sitios · 3 profundidades · 54 muestras secuenciadas

Variables ambientales

Humedad relativa del suelo · Temperatura · Elevación · pH

Datos

1.109 OTUs × 54 muestras
archivo: data/abundancias.tsv

Pregunta

¿Cómo afecta la aridez creciente a la diversidad y composición microbiana?

Lo que haremos

Regresión diversidad alfa vs. aridez · Composición vs. variables ambientales

El repositorio del curso

github.com/cristinamunoz/curso-analisis-claude

curso-analisis-claude/

├─ CLAUDE.md ← contexto del agente

├─ README.md ← enunciado del reto

├─ docs/

| └─ programa_curso_vJulio2026.pdf

| └─ pasos_curso_manana1.md

| └─ pasos_curso_manana2.md

├─ paper/

| └─ mSystems.00195-16.pdf

├─ data/

| └─ abundancias.tsv ← 1109 OTUs × 54

| └─ metadata.tsv ← 75 muestras × 21

└─ expected_outputs/ ← referencia

Tu branch personal

Clonas el repo, creas un branch con tu nombre — tu espacio de trabajo no toca el de los demás

data/ — solo lectura

abundancias.tsv y metadata.tsv no se modifican. Se leen con /add en cada sesión

CLAUDE.md + docs/

El CLAUDE.md es el contexto del agente. docs/ tiene el programa y las guías paso a paso de cada mañana



Ejercicio 1

¿Crear un archivo es lo mismo que describirlo?

Hazle la misma instrucción a claude.ai (navegador) y a Claude Code (terminal):

"Crea un archivo llamado fibonacci.txt con los primeros 10 números de Fibonacci y guárdalo en esta carpeta"

- ¿Cuál de los dos produjo el archivo? → Verifica con ls en la terminal
- ¿Qué hizo claude.ai en cambio? ¿Te dio el código para que tú lo corrieras?
- ¿Cuál es la diferencia entre describir una acción y ejecutarla?

5 min · el archivo existe o no existe — no hay respuesta intermedia

Agenda del día 1

Aprender a trabajar con Claude Code — sin escribir una línea de código

09:00

M1

¿Qué es un agente? Demo

09:30

M2

Instalación y entorno

11:00

M3

Modelos, tokens y contexto

11:30

M4

Manejo de archivos

12:00

M5

CLAUDE.md, Skills y MCP

12:30

M6

Buenas prácticas de prompting

Módulo 2

Instalación y entorno

La terminal, Claude Code y tu primer branch en GitHub

09:30 – 10:30 · 60 minutos

¿Por qué GitHub en un curso de IA?

No es solo almacenamiento — es el eje de reproducibilidad del análisis

En ciencia asistida por IA, la reproducibilidad ya no depende solo del código — depende también de los prompts, el contexto del agente y las decisiones tomadas en cada sesión. GitHub guarda todo eso.

Control de versiones del análisis

Cada commit guarda el estado de tu CLAUDE.md, prompts y resultados — como un cuaderno de laboratorio con fecha y firma.

Branches = espacios de trabajo independientes

Cada participante trabaja en su propio branch sin pisar el código de los demás. Mismo repositorio, soluciones distintas — el experimento central del curso.

Comparación entre soluciones

En el Módulo 10, compararemos branches en vivo: mismo paper, mismo dataset, distintos CLAUDE.md y prompts → resultados distintos. El branch es la variable experimental.

Reproducibilidad real

Cualquier persona puede clonar tu branch y reproducir exactamente tu análisis: mismo agente, mismo contexto, mismo resultado.

La terminal sin miedo

Solo 3 comandos — los mismos que usaremos todo el curso

```
pwd
```

¿Dónde estoy?

Muestra la ruta de la carpeta donde estás parado

```
ls
```

¿Qué hay aquí?

Lista los archivos y carpetas del directorio actual

```
cd nombre/
```

Moverme a una carpeta

Entra a la carpeta 'nombre'. Usa `cd ..` para subir un nivel

Tip: usa la tecla Tab para autocompletar nombres de archivos y carpetas

Autenticar Claude Code

Con Node.js ya instalado desde los requisitos previos

```
# Verificar Node.js  
  
node --version  
  
# Instalar Claude Code (si no está aún)  
  
npm install -g @anthropic-ai/claude-code
```

```
# Iniciar Claude Code  
  
claude  
  
# Primera vez: pedirá login con Claude Pro
```

Qué ocurre al ejecutar claude

1. Se abre una sesión interactiva en la terminal
2. Primera vez: redirige al navegador para iniciar sesión con Claude Pro
3. Una vez autenticado, verás el prompt > listo para instrucciones
4. Para salir: /exit o Ctrl+C

Checkpoint: escribe 'hola' en Claude Code y verifica que responde antes de continuar con Git

Git esencial: clonar y crear tu branch

Solo 4 comandos — no es necesario saber más por ahora

```
# 1. Clonar el repositorio del curso

git clone https://github.com/cristinamunoz/curso-analisis-claude

# 2. Entrar a la carpeta

cd curso-analisis-claude

# 3. Crear tu branch personal

git checkout -b tu-nombre

# 4. Verificar que estás en tu branch

git branch
```

¿Qué es un branch?

Una copia de trabajo personal dentro del mismo repositorio. Cada participante tiene la suya — sin pisar el trabajo de los demás.

Verificación final del módulo

git branch debe mostrar tu nombre con * adelante y Claude Code debe responder al ejecutar claude.



Ejercicio 2

Checkpoint: tu entorno funcionando

Abre Claude Code en la carpeta del repositorio del curso
y haz esta pregunta:

- ¿Qué archivos ves en este directorio?
- ¿Puedes leer data/metadata.tsv y decirme cuántas muestras tiene y qué columnas ambientales hay?

Módulo 3

Modelos, tokens y contexto

Entender al agente por dentro para usarlo de forma eficiente

11:00 – 11:30 · 30 minutos · después del break

¿Qué es un token?

La unidad básica con la que Claude lee y procesa — y por qué el análisis complejo cuesta más

1 token $\approx$ $\frac{3}{4}$ de palabra en español. Claude cobra tokens por leer TU prompt y por escribir su respuesta — más contexto = más tokens = más lento y más costoso.

Análisis simple

```
"Calcula Bray-Curtis entre muestras"
```

Prompt: ~8 tokens

Datos en contexto: ~4.000 tokens

Total: ~4.008 tokens

 Una sesión de trabajo normal

Análisis multicapa

```
"Calcula Bray-Curtis sobre abundancias.tsv  
normalizado por rarefacción, ponderado por  
filogenia, con covariables de pH, temperatura  
y humedad de metadata.tsv, filtra OTU >10%"
```

Prompt: ~55 tokens

Datos en contexto: ~4.000 tokens

Metadata + filogenia: ~8.000 tokens

Total: ~12.055 tokens



Consumo 3x más — planifica qué incluyes

Economizar tokens: tu cuenta Claude Pro

Cuánto tienes, cuándo se reinicia y cómo sacarle el máximo

Tu cuenta Claude Code Pro

Límite de uso que se reinicia automáticamente cada ~5 horas · Sin límite de sesiones, pero con límite de tokens por ventana · Modelos Sonnet y Opus disponibles con /model

01

Un proyecto por sesión

Nunca mezcles dos análisis en la misma sesión. Si necesitas cambiar de proyecto, usa /clear primero.

02

Carga solo lo que necesitas

No hagas /add de todos los archivos al inicio. Añade solo el archivo que vas a usar en ese momento.

03

Prompts quirúrgicos

En vez de 'analiza todo el dataset', di exactamente qué columnas, qué test y qué output quieres. Menos tokens, mejor resultado.

04

/clear antes de cambiar de tarea

Al terminar un bloque de análisis, limpia el contexto. No acumules tokens de conversaciones anteriores.

05

Planifica antes de cargar archivos

El archivo de abundancias (1109 OTUs × 54 muestras) consume miles de tokens solo al cargarlo. Cárgalo cuando lo vayas a usar, no al inicio.

06

Usa Haiku para preguntas rápidas

Para verificar si un archivo existe, contar filas o explorar estructura: usa Haiku (/model). Guarda Sonnet para análisis reales.

Buenas prácticas en Claude Code

Hábitos que hacen la diferencia entre un análisis reproducible y uno caótico

Plan primero, siempre

Antes de cualquier análisis, pide al agente que te dé un plan por escrito y espere tu confirmación. Esto previene que ejecute cosas que no querías.

`/clear` — Limpiar el contexto

Usa `/clear` al cambiar de proyecto o cuando el agente empiece a dar respuestas que no tienen sentido. Reinicia la conversación sin cerrar Claude Code.

Verificar antes de continuar

Después de cada análisis, pide al agente: '¿Tiene sentido biológico este resultado?' y compara manualmente con `expected_outputs/`. No continúes si algo se ve extraño.

Un análisis por prompt

No pidas al agente que haga tres cosas a la vez. Un prompt bien definido = un output verificable. Si combinas tareas, es más difícil detectar dónde algo salió mal.

La ventana de contexto

El riesgo real: contaminar un proyecto con información de otro

 *El problema más silencioso de trabajar con agentes en sesiones largas no es que el agente olvide — es que mezcle contexto de dos proyectos distintos y lo aplique donde no debe.*

Escenario real:

Estás trabajando en el análisis de Atacama (Proyecto A) en una sesión de Claude Code. Sin cerrar la sesión, empiezas a trabajar en un análisis de microbioma intestinal con datos clínicos (Proyecto B). El agente recuerda que las muestras se llaman 'BAQ' y 'YUN', que las variables clave son 'AvgSoilRH' y 'temperatura', y las aplica al nuevo proyecto.

Nombres de variables incorrectos

El agente busca 'AvgSoilRH' en tu dataset clínico — esa columna no existe y genera un error silencioso o usa la columna más parecida.

Hipótesis del proyecto equivocado

El agente sigue asumiendo que la variable independiente es la aridez. Propone análisis correctos para Atacama pero incorrectos para datos clínicos.

Rutas de archivos cruzadas

Guarda outputs del Proyecto B en la carpeta del Proyecto A, o lee el archivo de abundancias de Atacama cuando debería leer el nuevo.

Elegir el modelo con /model

No todos los modelos son iguales — ni necesitas el mismo para todo

```
# Dentro de Claude Code:
```

```
/model
```

★ Recomendado para comenzar

Claude Sonnet

Úsalo cuando: Tarea del día a día

Velocidad equilibrada con calidad. Ideal para explorar datos, generar figuras, corregir errores. Recomendado como punto de partida.

Claude Opus

Úsalo cuando: Tareas complejas

Máxima capacidad de razonamiento. Usar cuando Sonnet no logra resolver un error estadístico persistente o una lógica de análisis difícil.

Claude Haiku

Úsalo cuando: Tareas simples y rápidas

Muy rápido y económico. Para verificar estructura de archivos, contar filas o preguntas exploratorias rápidas. No recomendado para análisis.

Ejercicio 3

Sonnet vs. Haiku: razonamiento estadístico real

Cambia al modelo que te toque (/model) y hazle esta pregunta a ambos:

"Obtuve estos resultados de PERMANOVA: $F=2.3$, $R^2=0.08$, $p=0.048$, con 4 grupos de $n=5$ muestras cada uno y 999 permutaciones. ¿Es este resultado confiable para publicación? ¿Qué problemas ves?"

- ¿Haiku notó que $R^2=0.08$ es muy bajo (solo 8% de varianza explicada)?
- ¿Sonnet señaló que $n=5$ da solo $5^4=625$ permutaciones posibles — cerca del límite para 999?
- ¿Alguno cuestionó el $p=0.048$ borderline con esas restricciones estadísticas?

Módulo 4

Manejo de archivos y entorno

Qué ve Claude Code y cómo darle acceso a los datos correctos

11:30 – 12:00 · 30 minutos

El directorio de trabajo

La diferencia entre 'tengo el archivo' y 'el agente lo puede ver'

✗ Sin directorio correcto

```
# El agente responde:
```

```
No encuentro atacama_abundancias.csv
```

Diagnóstico: ejecuta `pwd` en la terminal. Si no muestra la carpeta del repo, usa `cd`.

✓ Con directorio correcto

```
# El agente responde:
```

```
Veó 54 muestras y 1109 OTUs...
```

Siempre ejecuta `claude` desde dentro de la carpeta del repositorio. Verifica con `pwd` antes de empezar.

💡 ¿Proyecto nuevo? Usa `claude init`

Ejecuta `claude init` en la carpeta vacía de tu nuevo proyecto. Claude Code creará automáticamente un `CLAUDE.md` plantilla con las secciones básicas ya definidas — el punto de partida oficial para cualquier proyecto nuevo.

Agregar archivos y configurar el proyecto

Dos comandos esenciales: /add para datos y claude init para nuevos proyectos

```
# Añadir archivos al contexto

/add data/metadata.tsv

/add data/abundancias.tsv

/add paper/mSystems.00195-16.pdf

# Ver qué archivos están en contexto

/files
```

```
# Para un proyecto completamente nuevo:

mkdir mi-nuevo-proyecto && cd mi-nuevo-proyecto

claude init

# Crea CLAUDE.md con plantilla + estructura
```

claude init hace por ti:

- ✅ Crea CLAUDE.md con secciones base ya definidas
- ✅ Pregunta el nombre del proyecto y el objetivo
- ✅ Sugiere estructura de carpetas
- ✅ Define convenciones de output por defecto
- ⚡ Punto de partida oficial — no partes de cero

Para este curso: el repo ya tiene un CLAUDE.md base. Usa claude init solo si en el futuro creas un proyecto nuevo desde cero — es la forma

Ejercicio 4

Explorar el dataset de Atacama

Desde tu branch, añade el metadata al contexto con `/add` y pide al agente:

- ¿Cuántas muestras hay por transecto (Baquedano vs. Yungay)?
- ¿Qué columnas de variables ambientales tiene el archivo?
- ¿Cuál es el rango de valores de AvgSoilRH?

Módulo 5

CLAUDE.md, Skills y conectores MCP

Darle al agente memoria, instrucciones permanentes y acceso a fuentes externas

12:00 – 12:30 · 30 minutos

CLAUDE.md: el briefing del agente

El archivo que Claude Code lee automáticamente al iniciar cada sesión

Sin CLAUDE.md, el agente empieza cada sesión sin saber nada sobre ti ni tu proyecto. Con CLAUDE.md, empieza exactamente donde lo dejaste.

```
# CLAUDE.md — estructura base

## Quien soy

Soy ecóloga microbiana. No sé programar.

## El proyecto

Analizar microbioma de Atacama (Neilson 2017).

## Los datos

data/abundancias.tsv  (1109 OTUs × 54)

data/metadata.tsv     (75 muestras × 21 vars)

## Reglas del agente

Antes de ejecutar: hazme un plan y espera.
```

Quién soy

Tu rol y formación — para que el agente calibre el nivel de explicación

El proyecto

Qué problema científico y con qué paper

Los datos

Qué archivos existen y cuáles son de solo lectura

Reglas del agente

Plan previo, guardar en outputs/, reportar errores

Demo: con y sin CLAUDE.md sobre la misma pregunta

¿Qué es una skill en Claude Code?

Instrucciones especializadas que el agente activa para tareas concretas

Una skill es un archivo .md con instrucciones para una tarea específica. A diferencia del CLAUDE.md — que se lee en cada sesión — una skill se activa solo cuando el agente detecta que la tarea coincide con su 'trigger'. Puedes tener muchas skills pero el agente te carga solo la que necesita en ese momento.



Skill de análisis

Trigger: 'análisis de diversidad'

Pasos exactos para un tipo de análisis: qué calcular primero, qué verificar, qué índices reportar. Garantiza que siempre uses la misma metodología en todo el proyecto.



Skill de visualización

Trigger: 'genera una figura'

Estándares de figuras: formato PNG/PDF, resolución 300 dpi, paleta viridis, carpeta de destino outputs/. Produce figuras consistentes sin repetir las instrucciones cada vez.



Skill de revisión

Trigger: 'revisa estos resultados'

Preguntas de validación automáticas: ¿tiene sentido biológico? ¿el R^2 es razonable? ¿coincide con expected_outputs? El agente la activa antes de terminar el análisis.



Skill de pipeline

Trigger: 'corre el pipeline completo'

Flujo de múltiples pasos encadenados: carga → limpieza → análisis → figura → exportación. Se activa con una sola instrucción y ejecuta todo el flujo en orden.

Skills: instrucciones modulares reutilizables

Cómo se diferencian de un CLAUDE.md normal

CLAUDE.md = contexto permanente del proyecto. Skill .md = instrucción modular activada para una tarea específica. La diferencia está en la estructura del archivo.

CLAUDE.md (contexto del proyecto)

```
## Quien soy

Soy ecóloga microbiana...

## Los datos

data/abundancias.tsv

## Reglas

Plan antes de ejecutar.

(prose libre, sin frontmatter)
```

skill-diversidad.md (skill específico)

```
---

name: Análisis de diversidad

trigger: análisis diversidad

---

## Pasos

1. Resumen estadístico primero

2. Boxplot antes de regresión

## Output esperado

Tabla índices + figura PNG 300dpi
```

Conectores MCP

Cómo conectar Claude Code a fuentes de datos externas

MCP (Model Context Protocol) permite conectar Claude Code a sistemas externos sin copiar y pegar datos manualmente.

Lo que ya funciona hoy

- Leer repositorios de GitHub directamente
- Acceder a documentación online en tiempo real
- Conectarse a sistemas internos de laboratorio

Para bioinformática en el futuro

- Consultar NCBI y descargar por accession number
- Acceder a tablas de EBI o QIITA directamente
- Conectar a pipelines en la nube (AWS, Galaxy)

¿Por qué importa ahora?

- Hoy lo usaremos conceptualmente con una demo
- El curso avanzado incluye configuración de MCPs propios
- Es la dirección hacia la que va toda la bioinformática con IA

Claude para ciencia: el ecosistema en crecimiento

MCPs y skills especializados para investigación científica

Anthropic y la comunidad científica están construyendo un ecosistema de herramientas especializadas que convierten a Claude Code en un asistente de investigación completo.

MCP · Literatura científica

PubMed, Semantic Scholar y arXiv: buscar papers por pregunta, descargar abstracts, extraer métodos y comparar resultados — sin salir de la terminal.

MCP · Bases de datos biológicas

NCBI (genes, secuencias, taxonomía), UniProt (proteínas), KEGG (rutas metabólicas). Consulta directa por accession number o término.

MCP · Herramientas de análisis

Conexión a R/Bioconductor, Galaxy, Snakemake. Ejecutar pipelines completos desde Claude Code y recibir los resultados de vuelta.

Skill · Análisis de microbioma

Flujo completo: carga de OTU table → rarefacción → diversidad alfa/beta → visualización → exportación. Activado con una sola instrucción.

Skill · Modelado metabólico

TFA/FBA sobre modelos SBML: carga, validación termodinámica, análisis de flujos, exportación de resultados. Directamente relevante para el paper pyTFA.

Skill · Estadística ecológica

PERMANOVA, NMDS, regresiones de diversidad. Con outputs estandarizados y verificación automática de supuestos estadísticos.

Ejercicio 5

Escribe tu CLAUDE.md personal

Crea el archivo CLAUDE.md en la raíz de tu branch. Debe tener:
Quién soy · El proyecto · Los datos · Reglas del agente · Al menos 1 skill

- Reinicia Claude Code y verifica: pregunta '¿Cuál es el objetivo de este proyecto?'
- El agente debe mencionar Neilson 2017, Atacama y la pregunta de aridez
- Añade el skill de figuras: PNG 300 dpi, guardar en outputs/, paleta viridis

Módulo 6

Buenas prácticas de prompting

Cómo dirigir al agente para obtener resultados precisos y reproducibles

12:30 – 13:00 · 30 minutos

Pedir plan antes de ejecutar

La instrucción más importante que puedes darle al agente

Sin esta instrucción

```
"Analiza la diversidad del dataset"
```

El agente ejecuta inmediatamente. Instala paquetes, corre código, genera archivos — y cuando termina, puede haber hecho algo completamente diferente a lo que esperabas.

Con la instrucción correcta

```
"Analiza la diversidad. PRIMERO hazme  
un plan y espera mi confirmación."
```

El agente describe qué va a hacer, qué paquetes necesita, qué archivos va a crear. Tú apruebas, corriges o rediriges antes de que ejecute una sola línea de código.

Añade esta instrucción a tu CLAUDE.md: Antes de ejecutar cualquier análisis, hazme un plan detallado y espera mi confirmación.

Contexto científico y verificación

El agente ejecuta — tú decides si el resultado tiene sentido

01

Dar contexto del dominio

✗ Haz una regresión de diversidad

✓ Haz una regresión de diversidad alfa (Shannon) vs. humedad relativa (AvgSoilRH). Hipótesis: a menor humedad, menor diversidad.

02

Pedir verificación científica

✗ ¿Listo?

✓ ¿Tiene sentido biológico este resultado? Si la correlación es negativa (más humedad, más diversidad), es coherente con Neilson 2017.

03

Definir el formato del output

✗ Genera la figura

✓ Scatter plot. Eje X: AvgSoilRH (%). Eje Y: Shannon. Color por transecto. Línea de regresión con banda de confianza 95%. PNG 300 dpi en outputs/.

Cuando el agente falla

Cómo instruirlo para que reporte, no que silencie

```
# Añadir a tu CLAUDE.md:
```

```
Si encuentras un error, muéstramelo completo y explícame en lenguaje simple qué significa.
```

```
No intentes corregirlo más de 2 veces sin preguntarme primero.
```

Error de Python

Causa: Paquete no instalado, sintaxis incorrecta, nombre de columna mal escrito

Qué hacer: El agente lo muestra, explica la causa y propone instalación con conda install. Tú apruebas.

Resultado sin sentido

Causa: El agente calculó algo correcto pero biológicamente absurdo (p. ej. $R^2 = -0.3$)

Qué hacer: NO es un error de código — es señal de que el análisis necesita revisión. El agente no puede detectarlo solo: eres tú quien debe revisarlo.

Alucinación estadística

Causa: El agente inventa un resultado o usa una función que no existe en Python

Qué hacer: Verifica SIEMPRE los números contra `expected_outputs/`. Si algo no coincide, pide al agente que muestre el código exacto que usó.

Ejercicio 6

Prompt vago vs. prompt estructurado

Con el dataset de Atacama en contexto, haz la misma solicitud de dos formas:

- Versión A: 'Analiza la diversidad del dataset'
- Versión B: 'Antes de ejecutar, hazme un plan. Calcula Shannon por muestra. Regresiona Shannon vs. AvgSoilRH. Hipótesis: a menor humedad, menor diversidad.'
- ¿Cuál requiere menos correcciones? ¿Cuál produjo un plan antes de ejecutar?

Al cierre del día 1

Tienes todo lo que necesitas para mañana

- ✓ Entorno instalado, Claude Code autenticado y branch en GitHub
- ✓ Sabes por qué GitHub es fundamental para ciencia reproducible con IA
- ✓ Sabes cambiar de modelo con /model y economizar tokens
- ✓ Entiendes el riesgo de contaminación de contexto entre proyectos
- ✓ Sabes usar /add, /clear, /memory y claude init
- ✓ Tu CLAUDE.md tiene contexto del proyecto Atacama con al menos 1 skill
- ✓ Sabes pedir plan antes de ejecutar y verificar resultados

Mañana

Día 2 · 09:00

El paper de Neilson, el dataset de Atacama, y la pregunta real:

¿Cómo afecta la aridez a la comunidad microbiana del suelo?